CLOUD COMPUTING AND AMAZON WEB SERVICES

A Structured Technical Reference for Retrieval-Augmented Generation (RAG) Testing

Purpose

This guide is designed as a test corpus for document-grounded question answering. It includes definitions, service distinctions, architectural examples, troubleshooting scenarios, and design exercises that support direct retrieval, multi-hop reasoning, and abstention tests.

How to use it

Upload the PDF to your RAG pipeline, ask questions that require exact facts or combining sections, and verify that answers are supported by retrieved passages. Include questions whose answers are not stated in the document to test whether the system avoids guessing.

Technical learning material • English edition • Prepared October 2026

Table of Contents

1. Cloud Computing: Core Concepts
2. AWS Global Infrastructure and Account Structure
3. Identity and Access Management
4. Networking with Amazon VPC
5. Compute Services
6. Storage Services
7. Databases and Data Services
8. Application Integration and Messaging
9. Security, Encryption, and Governance
10. Monitoring, Logging, and Operations
11. Reliability, Backup, and Disaster Recovery
12. Cost Management and Optimization
13. Infrastructure as Code and Deployment
14. Data Engineering and Machine Learning on AWS
15. Architecture Patterns and Worked Examples
16. Troubleshooting Scenarios and Decision Rules
17. Reference Glossary and Key Distinctions
18. Review Checklist and Design Exercises

Appendix A. RAG Evaluation Questions

Appendix B. Answer Key and Evidence Notes

Note: Service features, quotas, and pricing can change. Verify current AWS documentation before using this guide as an implementation specification.

1. Cloud Computing: Core Concepts

1.1 What cloud computing means

Cloud computing is a model for providing on-demand access to shared computing resources—such as servers, storage, databases, networking, analytics, and software—over a network. Resources can be provisioned quickly and released when they are no longer needed. Instead of purchasing every server in advance, an organization can obtain capacity from a provider and pay according to its usage or subscription.

A useful working definition has five characteristics: on-demand self-service, broad network access, resource pooling, rapid elasticity, and measured service. On-demand self-service means a customer can provision a resource without waiting for a provider employee to perform each step. Broad network access means services are reachable through standard network mechanisms from suitable clients. Resource pooling means provider resources serve multiple customers while maintaining logical separation. Rapid elasticity means capacity can expand or contract as demand changes. Measured service means usage is monitored and can be metered.

1.2 Service models

Infrastructure as a Service (IaaS) supplies fundamental computing resources such as virtual machines, networks, and disks. The customer generally manages the operating system, middleware, applications, and data. Amazon EC2 is a common AWS example. Platform as a Service (PaaS) provides a managed environment for deploying applications while the provider handles more of the underlying platform. AWS Elastic Beanstalk is an example of a service that helps deploy and manage applications. Software as a Service (SaaS) delivers a complete application to end users; the customer usually configures the application rather than managing its servers.

The labels are useful but not absolute. A managed database, for example, removes much of the routine database infrastructure work while the customer remains responsible for schema design, query behavior, access permissions, and data quality. Responsibility depends on the particular service and configuration, not just the service-model label.

1.3 Deployment models

A public cloud provider offers services over provider-operated infrastructure to many customers. A private cloud is dedicated to one organization and may be operated internally or by a third party. A hybrid approach connects or coordinates private infrastructure with public cloud resources. A multi-cloud approach uses services from more than one cloud provider. These approaches can coexist: a company might keep a regulated legacy database in a private environment while running a public-facing web tier on AWS.

Cloud adoption is not automatically cheaper or safer. Cost depends on architecture, utilization, data transfer, licensing, and operational discipline. Security depends on identity controls, configuration, monitoring, patching, and data handling. Moving an inefficient application to cloud infrastructure without redesign can preserve its inefficiencies and add new costs.

1.4 Regions and Availability Zones

An AWS Region is a separate geographic area that contains multiple Availability Zones. An Availability Zone (AZ) consists of one or more discrete data centers with independent power, networking, and connectivity, designed to be isolated from failures in other AZs. AZs within a Region are connected through high-bandwidth, low-latency networking. Region selection affects latency, service availability, regulatory obligations, and price.

A multi-AZ design can improve resilience against an individual AZ failure, but it does not by itself protect against every regional event, application bug, data deletion, or compromised credential. Cross-Region replication or backups may be needed for regional disaster recovery. AWS service availability and supported features differ by Region, so a design should verify the required service in its target Region before deployment.

2. AWS Global Infrastructure and Account Structure

2.1 AWS accounts and organizational boundaries

An AWS account is a fundamental boundary for resources, billing, quotas, and access control. A workload can be isolated in a dedicated account to reduce the blast radius of mistakes and make costs easier to attribute. AWS Organizations allows multiple accounts to be centrally managed. Organizational units (OUs) group accounts for applying governance policies, while service control policies (SCPs) define the maximum available permissions for accounts in an organization; an SCP does not grant permissions by itself.

A practical organization might separate production, development, security, logging, and shared networking into different accounts. This separation can reduce accidental changes to production and help enforce distinct access policies. The exact account structure should reflect organizational size, regulatory requirements, team boundaries, and operational maturity.

2.2 Edge locations and content delivery

AWS operates edge locations used by services such as Amazon CloudFront to cache and deliver content closer to viewers. A content delivery network (CDN) can reduce latency and origin load for cacheable content. CloudFront can deliver static assets and dynamic content, but cache behavior depends on cache policies, headers, query strings, cookies, and origin configuration. Sensitive or personalized responses must not be cached indiscriminately.

AWS Local Zones extend selected AWS services closer to large population or industry centers for applications that need low latency. AWS Wavelength embeds AWS infrastructure in telecommunications networks for certain mobile edge workloads. These options are not substitutes for every Region service: service availability and capacity vary by location.

2.3 Choosing a Region

Region selection should consider where users are located, data residency obligations, disaster-recovery requirements, service availability, latency, and price. A region close to users may improve response times, but moving data between Regions can introduce transfer costs and replication complexity. A recovery Region should be sufficiently independent for the intended failure scenario and should have the services, quotas, keys, and operational procedures required during recovery.

For example, an online retailer whose main users are in Europe might select a European Region for low latency and relevant data-residency requirements, then maintain backups or replicated data in a second Region if its recovery objectives justify the extra cost. This is an architectural example, not a claim that one Region is best for every European workload.

3. Identity and Access Management

3.1 Users, roles, and policies

AWS Identity and Access Management (IAM) controls authentication and authorization for AWS resources. An IAM policy is a JSON document describing allowed or denied actions on resources, optionally under conditions. An IAM user represents a long-term identity; an IAM role is an identity that can be assumed to obtain temporary credentials. Human users commonly access AWS through federated identity and roles rather than maintaining long-lived access keys.

The principle of least privilege means granting only the permissions needed for a task. A developer who needs to read logs should not automatically receive permission to delete production databases. Permissions should be reviewed as responsibilities change, and overly broad wildcard permissions should be avoided when practical.

3.2 Policy evaluation basics

In general, an explicit Deny overrides an Allow. An identity-based policy can grant permissions to a principal, while a resource-based policy can grant access directly to specified principals for supported resources. Permissions boundaries and organization service control policies can limit the maximum permissions available. A permission existing in one policy does not guarantee access if another applicable control denies the action.

For example, a role may have an identity policy allowing s3:GetObject on a bucket, but a bucket policy with an applicable explicit Deny can still prevent the read. Troubleshooting should inspect the principal, action, resource ARN, conditions, resource policy, permissions boundary, organization policies, and any relevant service-specific controls.

3.3 Temporary credentials and federation

Temporary security credentials have a limited lifetime and are issued through AWS Security Token Service (STS). They reduce the need to distribute permanent access keys. IAM roles can be assumed by AWS services, users, or external identities when the trust policy allows it. A trust policy answers who may assume the role; the role's permissions policies answer what the resulting session may do.

A workload running on Amazon EC2 should normally use an attached IAM role through an instance profile instead of storing access keys in application files. Applications can retrieve temporary credentials through the supported credential provider chain. Credentials should never be committed to source control, embedded in public images, or copied into logs.

3.4 Multi-factor authentication and access hygiene

Multi-factor authentication (MFA) requires an additional factor beyond a password. MFA is especially important for privileged identities and account-root access. The root user should be protected with strong authentication, used only for tasks that require it, and not used for everyday administration. Root access keys should not be created for routine workloads.

Access hygiene includes removing unused identities, rotating or eliminating long-lived keys, reviewing console sign-in activity, setting appropriate password and federation policies, and alerting on suspicious activity. MFA is not a replacement for least privilege, network controls, or monitoring; it is one layer in a broader identity security strategy.

4. Networking with Amazon VPC

4.1 Virtual Private Cloud fundamentals

Amazon Virtual Private Cloud (Amazon VPC) provides a logically isolated virtual network for AWS resources. A VPC is associated with an IPv4 CIDR block and may also use IPv6 addressing. Subnets are subdivisions of a VPC CIDR and are each associated with a single Availability Zone. A subnet is considered public when its route table has a route to an internet gateway and resources have appropriate public addressing and security configuration; the label alone does not make every resource internet-accessible.

Route tables determine where network traffic is directed. A route such as 0.0.0.0/0 is a default route for IPv4 traffic. An internet gateway enables communication between a VPC and the internet when routing and addressing are configured correctly. A NAT gateway allows resources in a private subnet to initiate outbound IPv4 connections without accepting unsolicited inbound connections through that NAT gateway.

4.2 Security groups and network ACLs

A security group acts as a stateful virtual firewall attached to supported network interfaces. It evaluates inbound and outbound rules; response traffic for an allowed connection is tracked automatically. A network access control list (network ACL) operates at the subnet boundary and is stateless, so return traffic must be permitted by rules as well. Network ACL rules are evaluated in numerical order, with the first matching rule applied.

A typical web architecture might permit inbound HTTPS on TCP port 443 to a load balancer, permit the load balancer to reach application instances on an application port, and permit the application tier to connect to a database only on the database port. The database should not accept connections directly from the public internet. Security groups can reference other security groups to express tier-to-tier access more maintainably than broad IP ranges.

4.3 VPC connectivity options

VPC peering connects two VPCs privately, but it is not transitive: a VPC cannot use one peering connection to route through the peer to a third VPC. AWS Transit Gateway provides a central hub for connecting many VPCs and supported networks. AWS Site-to-Site VPN establishes encrypted connectivity between a VPC and a customer network over IPsec. AWS Direct Connect provides a dedicated network connection from a customer location to AWS; encryption should be designed separately when required.

Private connectivity to AWS services may use VPC endpoints. Gateway endpoints are available for Amazon S3 and DynamoDB. Interface endpoints use AWS PrivateLink and elastic network interfaces for many supported services. Endpoints can reduce dependence on internet routing and help keep service traffic on private paths, but policies and DNS configuration still determine whether requests succeed.

4.4 DNS and load balancing

Amazon Route 53 is a scalable DNS service that can register domains, route DNS queries, and perform health checks. DNS routing policies include simple, weighted, latency-based, failover, geolocation, geoproximity, and multivalue answer routing. A DNS health check does not guarantee that an entire application is healthy; checks should target a meaningful endpoint and be paired with monitoring and recovery procedures.

Elastic Load Balancing distributes incoming traffic across registered targets. An Application Load Balancer operates at the application layer and supports HTTP/HTTPS routing features. A Network Load Balancer operates at the transport layer and supports high-performance TCP, UDP, and TLS use cases. Load balancing improves distribution and can support high availability, but the application targets, dependencies, and scaling policies must also be designed for failure.

5. Compute Services

5.1 Amazon EC2

Amazon Elastic Compute Cloud (Amazon EC2) provides resizable virtual servers called instances. An Amazon Machine Image (AMI) supplies the information required to launch an instance, including an operating system and software configuration. Instance types differ in CPU, memory, storage, and network characteristics. Choosing an instance should be based on workload measurements rather than simply selecting the largest available size.

EC2 purchasing options include On-Demand Instances, Savings Plans or Reserved Instance arrangements for eligible use cases, and Spot Instances that use spare capacity and may be interrupted. Spot is often suitable for fault-tolerant batch processing, distributed data analysis, or flexible workloads. It is risky for a single irreplaceable process without checkpointing or interruption handling.

5.2 Auto Scaling and load distribution

Amazon EC2 Auto Scaling can add or remove instances according to desired capacity, schedules, or scaling policies. Dynamic scaling can respond to metrics such as average CPU utilization or application-specific demand. A target tracking policy attempts to maintain a chosen metric near a target value. Scaling is not instantaneous; warm-up periods, health checks, capacity limits, and application startup time affect behavior.

A resilient web tier often combines a load balancer with an Auto Scaling group spanning multiple Availability Zones. The load balancer routes requests only to healthy targets, while Auto Scaling replaces unhealthy instances and adjusts capacity. This design still requires the application to externalize or replicate state if requests may be handled by different instances.

5.3 AWS Lambda and serverless computing

AWS Lambda runs supported code in response to events without requiring the customer to manage server instances. It is commonly used for event processing, lightweight APIs, scheduled jobs, and file-processing workflows. Lambda functions have execution limits and concurrency considerations, and their duration, memory allocation, architecture, and invocation pattern affect cost. Cold starts can add latency when an execution environment must be initialized.

Serverless does not mean there are no servers; it means the provider manages more of the infrastructure lifecycle. A function should be designed for retries and duplicate events where relevant. Many event sources provide at-least-once delivery semantics, so idempotent processing is important. Long-running workloads or specialized runtime requirements may fit containers or EC2 better.

5.4 Containers: ECS, EKS, and Fargate

Containers package an application and its dependencies into a portable image. Amazon Elastic Container Service (Amazon ECS) is a managed container orchestration service. Amazon Elastic Kubernetes Service (Amazon EKS) is a managed Kubernetes service. AWS Fargate provides serverless compute for supported ECS and EKS workloads, removing the need to manage the underlying worker servers directly.

Containers are useful when teams need consistent environments, independent deployment units, or control over runtime dependencies. They still require image security, patching of base images, secrets management, logging, networking, resource limits, and deployment strategies. Fargate changes infrastructure management responsibility but does not eliminate application operations or cost governance.

6. Storage Services

6.1 Amazon S3 object storage

Amazon Simple Storage Service (Amazon S3) is object storage. Data is stored as objects inside buckets; an object has a key and associated metadata. S3 is appropriate for images, documents, backups, data lakes, static assets, and many analytics workflows. It is not a conventional mounted block disk, and applications should account for object API semantics and request patterns.

S3 storage classes support different access patterns and cost trade-offs. S3 Standard is designed for frequently accessed data. S3 Intelligent-Tiering can move eligible objects between access tiers based on observed access, subject to its rules and monitoring charges. S3 Standard-Infrequent Access is intended for less frequently accessed data that still needs rapid access. Archive classes can lower storage cost but introduce retrieval delays or minimum storage-duration considerations.

6.2 Durability, versioning, and lifecycle

Amazon S3 is designed for high durability, but durability is not the same as protection against accidental deletion, malicious changes, or application-level corruption. Versioning preserves multiple versions of objects and can help recover from overwrites or deletions. Lifecycle policies can transition objects to other storage classes or expire eligible objects according to age and rules. Object Lock can help protect objects from deletion or alteration for a defined retention period when configured appropriately.

A backup strategy should consider independent copies, retention, access restrictions, restore testing, and recovery objectives. A versioned bucket alone is not a complete disaster-recovery plan if an attacker can delete versions or if the account becomes inaccessible. Replication, separate accounts, and tightly controlled backup administration can reduce some of these risks.

6.3 EBS and EFS

Amazon Elastic Block Store (Amazon EBS) provides persistent block storage volumes for use with EC2. EBS volume types have different performance and cost profiles. EBS snapshots are point-in-time backups stored and managed through AWS infrastructure; they can be used to create new volumes. The attachment and availability characteristics of a volume depend on its type and configuration.

Amazon Elastic File System (Amazon EFS) is a managed elastic file system that can be mounted by supported compute services and accessed concurrently by multiple clients. It is often used when applications need shared file semantics. EBS is generally a block-storage choice associated with instance storage needs, while EFS is a shared file-storage choice. Neither should be selected solely by comparing price per gigabyte; performance, access pattern, throughput, and availability matter.

6.4 Choosing a storage type

Use object storage when data is naturally addressed as objects and accessed through APIs; use block storage when an operating system or application needs a block device; use file storage when multiple clients need a shared hierarchical file system. A data lake often stores raw and curated objects in S3, while a transactional database may use block storage under a managed service. Shared media processing may use EFS if the workload requires common file access.

Before choosing storage, record expected capacity, request rate, read/write ratio, latency needs, durability and recovery needs, retention rules, encryption requirements, and data transfer paths. Lifecycle policies and compression can reduce storage costs, but they should not violate retrieval-time requirements or legal retention obligations.

7. Databases and Data Services

7.1 Relational databases and Amazon RDS

A relational database organizes data into tables with defined relationships and uses SQL for queries and data manipulation. Amazon Relational Database Service (Amazon RDS) manages many routine database tasks for supported relational engines, such as provisioning, backups, patching options, and recovery features. Customers remain responsible for schema design, query efficiency, credentials, data modeling, and choosing appropriate availability and backup settings.

A Multi-AZ RDS deployment is designed to improve availability by maintaining a standby in another Availability Zone for supported configurations. Read replicas serve a different purpose: they can offload read traffic and, depending on the engine and configuration, support replication and reporting use cases. A read replica is not automatically equivalent to a Multi-AZ standby, and replication lag may affect read-after-write expectations.

7.2 Amazon DynamoDB

Amazon DynamoDB is a managed NoSQL database service supporting key-value and document data models. Tables contain items, and items contain attributes. Primary keys can be partition keys alone or partition keys combined with sort keys. Access patterns should guide key design because efficient queries generally depend on key conditions and appropriate indexes rather than scanning an entire table.

DynamoDB offers on-demand and provisioned capacity modes. Provisioned mode requires capacity planning and may use auto scaling; on-demand mode adjusts to traffic without the same explicit capacity provisioning, with its own pricing and throughput considerations. Applications should understand consistency options, partition-key distribution, item-size limits, indexes, and retry behavior before choosing DynamoDB.

7.3 Analytics and data warehousing

Amazon Redshift is a managed cloud data warehouse designed for analytical SQL workloads over large datasets. It differs from an operational relational database optimized for many small transactions. Amazon Athena allows SQL queries against data in sources such as S3 without requiring the user to provision a traditional database cluster. Query design, file format, partitioning, compression, and scanned data volume influence Athena performance and cost.

AWS Glue provides data integration capabilities, including cataloging and ETL workflows. The AWS Glue Data Catalog stores metadata about data assets and is commonly used by analytics services. A typical analytics pipeline may land raw events in S3, catalog datasets, transform data with Glue, query it with Athena, and publish curated results for dashboards or a warehouse.

7.4 Caching with Amazon ElastiCache

Amazon ElastiCache provides managed in-memory caching options, including Redis OSS-compatible and Memcached engines depending on current service offerings. Caching can reduce database load and improve response time for frequently accessed data. It introduces invalidation, expiration, consistency, and cache-miss behavior that application designers must address.

A cache-aside pattern checks the cache first, retrieves missing data from the database, then populates the cache. Time-to-live (TTL) settings can bound how long entries remain. A cache should generally not be treated as the sole durable source of truth unless the chosen architecture explicitly supports that role. Cache stampedes can occur when many requests simultaneously miss the same popular key.

8. Application Integration and Messaging

8.1 Amazon SQS

Amazon Simple Queue Service (Amazon SQS) is a managed message queuing service that decouples producers from consumers. A producer can place work on a queue while consumers process messages at their own pace. Standard queues offer high throughput and at-least-once delivery, so duplicates can occur and message order is not generally guaranteed. FIFO queues support ordering and deduplication features within their documented constraints.

A visibility timeout temporarily hides a received message while a consumer processes it. If the consumer does not delete the message before the timeout expires, it can become visible again. Dead-letter queues can isolate messages that repeatedly fail processing. Visibility timeout, retry policy, message retention, and idempotent handlers should be designed together.

8.2 Amazon SNS and EventBridge

Amazon Simple Notification Service (Amazon SNS) is a pub/sub messaging service that can fan out messages to multiple subscribers. Amazon EventBridge routes events from AWS services, supported SaaS sources, and applications to targets using event buses and rules. SQS is useful for buffering work for consumers; SNS is useful for fan-out notifications; EventBridge is useful for routing events based on event content and rules. These services can be combined.

For example, an order service can publish an event when an order is placed. EventBridge can route the event to fulfillment and analytics targets, while an SQS queue buffers fulfillment work. The design should account for retries, duplicate delivery, schema evolution, permissions, and failed targets. Event-driven architectures reduce direct coupling but make tracing and end-to-end debugging more important.

8.3 Step Functions and workflow orchestration

AWS Step Functions coordinates workflows across supported AWS services and application tasks. A state machine describes states, transitions, branching, retries, and error handling. It is useful when a process contains multiple steps with explicit success and failure paths, such as validating an uploaded document, running analysis, storing a result, and notifying a user.

Workflow orchestration can make process logic observable and easier to recover, but it does not remove the need to design idempotent tasks. A retry can repeat a side effect if the task completed but the workflow did not receive confirmation. Each state should have clear inputs, outputs, timeout behavior, retry limits, and compensating actions where a transaction cannot be rolled back atomically.

9. Security, Encryption, and Governance

9.1 Shared responsibility model

AWS security follows a shared responsibility model. AWS is responsible for security of the cloud, including the underlying global infrastructure, hardware, and foundational services. Customers are responsible for security in the cloud, with the precise division depending on the service. For EC2, customers manage the guest operating system, applications, and relevant configuration. For a managed service such as S3, AWS manages more of the underlying infrastructure, while customers still manage data, permissions, and configuration.

Moving to a managed service shifts some operational tasks to AWS but does not make the workload automatically secure. A public S3 bucket, an overly permissive IAM role, exposed credentials, or unencrypted sensitive data can still create customer-side risks. Teams should map responsibilities per service and document who owns identity, data classification, patching, incident response, and recovery.

9.2 Encryption and key management

Encryption in transit protects data as it moves between systems, commonly through TLS. Encryption at rest protects stored data, often using service-managed keys or AWS Key Management Service (AWS KMS) keys. KMS helps create and control cryptographic keys and integrates with supported AWS services. Key policies, IAM policies, grants, rotation settings, and administrative access influence who can use or manage keys.

Encryption does not replace access control. An authorized principal may still read decrypted data, and an overly broad key policy can undermine intended separation. Key deletion schedules and key disablement can make encrypted data unavailable, so key lifecycle management must be included in recovery planning. Applications should avoid logging plaintext secrets or sensitive payloads.

9.3 Threat detection and audit trails

AWS CloudTrail records account activity and API events for supported services, helping investigators understand who performed an action, when it occurred, and which resources were involved. CloudTrail event history provides recent management-event history, while trails and CloudTrail Lake support broader collection and analysis patterns according to their configurations. Logging should be planned for retention, integrity, access control, and cost.

Amazon GuardDuty analyzes supported data sources and activity to identify potential threats. AWS Security Hub provides a consolidated view of security findings and posture checks across supported integrations. Amazon Inspector helps identify software vulnerabilities and unintended network exposure for supported resources. These services can improve detection, but findings still require triage, remediation ownership, and incident-response procedures.

9.4 Secrets and compliance

AWS Secrets Manager helps store, retrieve, and rotate secrets such as database credentials and API keys. Systems Manager Parameter Store can store configuration values and, for supported configurations, secure parameters. A secret should be granted only to the identities that need it, and applications should retrieve it through approved mechanisms rather than hard-coding it in source files.

Compliance is a shared operational program, not a product checkbox. Organizations must determine applicable laws, contractual requirements, data classifications, evidence requirements, and retention obligations. AWS provides documentation, reports, and services that can support compliance work, but customers remain responsible for using services in a compliant way and validating their own controls.

10. Monitoring, Logging, and Operations

10.1 Amazon CloudWatch

Amazon CloudWatch collects metrics, logs, and events from AWS resources and applications. Metrics are time-ordered numerical data points, such as CPU utilization, request count, or queue depth. CloudWatch alarms evaluate metrics against thresholds or anomaly-detection models and can trigger notifications or automated actions. A metric without an agreed response plan may provide little operational value.

A useful alert should be actionable, have a clear owner, and indicate the impact or likely cause. For a queue-based application, queue age may be more informative than queue depth alone because a large queue can be acceptable during a brief burst but harmful when messages wait too long. Dashboards should combine infrastructure health with customer-facing indicators such as latency and error rate.

10.2 Logging and tracing

Logs provide event details; metrics summarize behavior over time; traces show how a request moves through distributed components. AWS X-Ray supports tracing for instrumented applications and supported integrations. Correlation identifiers help connect a user request across services. Sensitive data should be excluded or redacted from logs, and log access should be restricted because logs can contain personal or operationally sensitive information.

Structured logs with consistent fields—timestamp, severity, service, environment, request ID, and error category—are easier to query than unstructured text. Distributed systems need synchronized clocks and consistent trace context. Sampling can reduce tracing cost but may hide rare errors if poorly configured; tail-based or targeted sampling can help preserve high-value traces where supported by the chosen tooling.

10.3 Operational excellence

Operational excellence includes preparing for failures, documenting procedures, learning from incidents, and improving systems continuously. Runbooks describe repeatable actions for common events, while playbooks may guide more complex investigations. Game days and controlled failure exercises can test whether alerts, escalation paths, backups, and recovery mechanisms work as expected.

A post-incident review should focus on contributing conditions and system improvements rather than blame. Useful outputs include a timeline, customer impact, detection gap, recovery actions, and prioritized follow-up items with owners. Reliability improves when teams measure both technical outcomes and the effectiveness of their operational processes.

11. Reliability, Backup, and Disaster Recovery

11.1 Reliability concepts

High availability aims to keep a service operating despite component failures. Fault tolerance aims to continue operation despite specified failures, often through additional redundancy and complexity. Durability concerns the likelihood that stored data remains intact. These concepts are related but not interchangeable: a system can be highly available while still being vulnerable to accidental data deletion.

A single EC2 instance in one Availability Zone is a single point of failure. A load-balanced application tier across multiple AZs can tolerate some instance or AZ failures, provided its dependencies also support the design. A multi-AZ front end connected to a single-AZ database can still experience a database-related outage.

11.2 RTO and RPO

Recovery Time Objective (RTO) is the target maximum time to restore a service after disruption. Recovery Point Objective (RPO) is the target maximum amount of data loss measured in time. An RTO of four hours means the service should be restored within four hours under the defined scenario. An RPO of fifteen minutes means recovery should lose no more than roughly fifteen minutes of updates, subject to the system's measurement and recovery design.

Lower RTO and RPO targets generally require more investment in replication, automation, standby capacity, testing, and operational readiness. A business should define these objectives by workload and impact rather than selecting the smallest possible values for every system. Backup frequency alone does not prove that an RPO can be met; restore duration and consistency must also be tested.

11.3 Disaster recovery strategies

Common disaster recovery approaches range from backup-and-restore, through pilot light and warm standby, to multi-site active/active designs. Backup-and-restore is often less expensive but typically has longer recovery time. Pilot light keeps core data and selected infrastructure ready, with additional resources created during recovery. Warm standby maintains a scaled-down but functional environment. Active/active serves traffic from multiple locations and can provide fast recovery, but increases complexity and synchronization requirements.

A disaster recovery plan should specify triggers, decision authority, dependencies, DNS or traffic changes, data recovery steps, security controls, communication, and validation. Recovery testing should include realistic failures and confirm data integrity, not merely that a server starts. A plan that depends on an undocumented administrator or unavailable credential may fail when it is needed most.

12. Cost Management and Optimization

12.1 Understand cloud billing

Cloud costs can include compute time, storage capacity, requests, data transfer, managed-service capacity, support plans, and optional features. Data transfer between services, Availability Zones, or Regions can be a meaningful cost driver depending on architecture and current pricing. Pricing varies by Region and service configuration, so estimates should be checked against current AWS pricing information rather than relying on a universal rate.

AWS Cost Explorer helps analyze historical and forecasted spending. AWS Budgets can track cost or usage thresholds and notify designated recipients. Cost and Usage Reports provide detailed billing data for deeper analysis. Cost allocation tags and cost categories help attribute spend to teams, environments, applications, or business units when applied consistently.

12.2 Compute and storage optimization

Rightsizing compares actual utilization and performance needs with provisioned capacity. Stopping non-production instances outside working hours, using Auto Scaling, choosing appropriate purchasing options, and removing unused resources can reduce waste. Savings Plans offer discounted rates in exchange for a commitment to a consistent amount of eligible usage over a term. Spot capacity can be economical for interruptible workloads.

Storage optimization includes lifecycle transitions, retention policies, compression, deleting unneeded snapshots, and selecting suitable performance settings. These actions should be based on access patterns and recovery requirements. A lower storage price can be offset by retrieval charges or delays, and deleting an apparently old backup may violate a retention policy or remove a needed recovery point.

12.3 FinOps practices

FinOps is a collaborative practice that brings finance, engineering, and business teams together to make informed cloud spending decisions. It emphasizes visibility, accountability, optimization, and continuous measurement. A useful unit metric might be cost per completed order, cost per processed document, or cost per active user rather than total monthly spend alone.

Cost optimization should not be measured in isolation. Reducing compute spend by weakening redundancy may violate availability goals; reducing logs too aggressively may undermine incident response. Teams should evaluate changes against reliability, security, performance, and business outcomes. Cost reviews work best when they produce owned actions and track their effect over time.

13. Infrastructure as Code and Deployment

13.1 Infrastructure as Code

Infrastructure as Code (IaC) defines infrastructure in version-controlled files rather than relying only on manual console changes. AWS CloudFormation describes AWS resources in templates and provisions them as stacks. Terraform is a third-party IaC tool commonly used with AWS providers. IaC improves repeatability, reviewability, and environment consistency, but unsafe templates can reproduce mistakes quickly.

Good IaC practice includes code review, small changes, reusable modules, parameterization, secret handling, policy checks, and state protection where the tool uses state. Changes should be tested in a non-production environment when practical. Drift detection helps identify differences between declared configuration and deployed resources, but it does not replace review of intended changes or runtime monitoring.

13.2 CI/CD pipelines

Continuous integration (CI) automates checks when code changes, such as linting, unit tests, dependency scans, and build validation. Continuous delivery (CD) keeps software in a deployable state; continuous deployment automatically releases changes after the required checks pass. AWS CodeBuild builds and tests code, CodePipeline orchestrates stages, and CodeDeploy automates supported deployment workflows to supported compute targets.

A secure pipeline should use scoped roles, protected branches, artifact integrity controls, dependency scanning, and approval gates where appropriate. Production credentials should not be embedded in repositories or build scripts. A successful build proves that a defined set of checks passed; it does not prove that the release is free of defects or that operational rollback will succeed.

13.3 Deployment strategies

A rolling deployment gradually replaces old application instances with new ones. A blue/green deployment maintains separate environments and shifts traffic after validation. A canary deployment exposes a small share of traffic to a new version before increasing exposure. Each strategy has trade-offs in capacity, cost, rollback speed, and database compatibility.

A canary release might direct 5 percent of requests to a new version, monitor error rate and latency, then increase traffic if metrics remain within defined thresholds. The percentage is a design choice rather than a universal best practice. Rollback planning should account for database schema changes, message formats, and irreversible side effects; reverting application binaries alone may not restore the prior state.

14. Data Engineering and Machine Learning on AWS

14.1 Data ingestion and processing

A data pipeline moves information from source systems through ingestion, validation, transformation, storage, and consumption. Amazon Kinesis services support streaming data use cases, while AWS Glue can support batch and integration workflows. Amazon S3 is commonly used as a durable landing and curated storage layer. Pipeline design should define schema, event time, late-arriving data, duplicate handling, and data-quality rules.

For example, an online service may send click events to a streaming ingestion layer, store raw events in S3, transform them into partitioned columnar files, catalog the output, and query it using Athena. Partitioning by date can reduce scanned data when queries filter on that field. Excessively small files can increase overhead, so compaction may be useful.

14.2 Machine learning services

Amazon SageMaker provides tools and managed infrastructure for building, training, deploying, and operating machine-learning models. A model lifecycle may include data preparation, experiment tracking, training, evaluation, model registration, endpoint deployment, monitoring, and retraining. Model quality depends on representative data, appropriate evaluation, and monitoring for drift—not only on the hosting service.

Batch inference is appropriate when predictions can be generated asynchronously for a dataset. Real-time endpoints are useful when an application needs low-latency predictions on demand. Serverless inference and asynchronous inference options may fit particular traffic patterns, subject to current service support and limits. Model endpoints require security controls, observability, capacity planning, and cost monitoring.

14.3 Governance and data quality

Data quality dimensions include accuracy, completeness, consistency, timeliness, uniqueness, and validity. A dataset can be syntactically valid yet unsuitable for analysis if key fields are missing or labels are biased. Validation checks should run close to ingestion and transformation stages, with failures routed to a quarantine or review process rather than silently discarded.

Data governance defines ownership, classification, permitted use, retention, lineage, and access policies. AWS Lake Formation can help manage permissions for supported data lake resources, while the Glue Data Catalog stores metadata. Governance must include business definitions and accountability, not just technical permissions. Sensitive attributes should be handled according to applicable policy and law.

15. Architecture Patterns and Worked Examples

15.1 Highly available three-tier web application

Consider a web application with a presentation tier, an application tier, and a data tier. Route 53 can resolve the public domain to an Application Load Balancer. The load balancer sends HTTPS requests to application instances in private subnets across two or more Availability Zones. An Auto Scaling group adjusts instance capacity. A managed relational database uses a suitable Multi-AZ configuration, and static assets are stored in S3 and delivered through CloudFront.

Security groups allow public HTTPS only to the load balancer, application traffic only from the load balancer to the application tier, and database traffic only from authorized application resources. IAM roles grant the application limited access to required services. CloudWatch monitors latency, error rates, CPU, and database metrics. Backups and a documented restore test address data recovery. Each element contributes to resilience, but the entire architecture must be tested as a system.

15.2 Event-driven document processing

A user uploads a document to an S3 bucket. An event notification starts a workflow, which places a message on SQS or invokes a supported orchestration path. A Lambda function validates file type and size, records metadata, and starts a processing task. Results are stored in a separate prefix or bucket, and an event or notification informs the user that processing has completed.

Controls should include least-privilege roles, encryption, upload size limits, malware or content validation where required, idempotency, retry limits, and a dead-letter path for repeated failures. The workflow should avoid trusting the file extension alone. Monitoring should distinguish invalid user input from infrastructure failures, and lifecycle rules can remove temporary artifacts after the approved retention period.

15.3 Analytics dashboard architecture

A company collects application events and daily sales records. Raw files are stored in S3, metadata is cataloged, and transformations produce cleaned tables in a curated prefix. Athena supports ad hoc SQL analysis, while a warehouse such as Redshift may serve repeated, complex analytical workloads. A dashboard reads aggregated data from the selected query or serving layer.

The team should define data freshness objectives, quality checks, access policies, and ownership for each dataset. Partitioning and columnar formats can reduce query cost. Dashboard caching may improve responsiveness, but refresh intervals must match business needs. If the dashboard shows revenue, the definition of revenue, refunds, time zone, and late-arriving transactions should be documented to prevent misleading comparisons.

15.4 Choosing between EC2, containers, and Lambda

EC2 offers broad operating-system and runtime control and is useful for specialized software or workloads that need persistent instance-level configuration. Containers package dependencies and can run under ECS or EKS; Fargate reduces worker-node management for supported workloads. Lambda is attractive for event-driven tasks with supported runtimes and bounded execution characteristics. The correct choice depends on runtime needs, duration, traffic pattern, scaling behavior, portability, operational skills, and total cost.

A small image-thumbnail function triggered by S3 may fit Lambda. A long-running API with several cooperating services may fit containers. A legacy application requiring a specialized operating system configuration may fit EC2. These are starting points, not rigid rules; benchmark the workload and evaluate service limits, security responsibilities, observability, and deployment complexity.

16. Troubleshooting Scenarios and Decision Rules

16.1 A website is unreachable

Begin by separating DNS, network, load-balancer, application, and dependency failures. Confirm that the domain resolves to the expected target and that the certificate is valid. Check listener rules, target health, security-group rules, subnet routes, and network ACLs. Inspect application logs and health-check paths. Verify that the application binds to the expected interface and port, and that upstream dependencies are reachable.

If a target is unhealthy, changing DNS may not solve the underlying issue. A security group is stateful, whereas a network ACL is stateless. A public subnet route to an internet gateway does not by itself provide a private instance with public reachability. Troubleshooting should test each layer and record the observed evidence before changing multiple settings at once.

16.2 An application cannot read an S3 object

Check the identity used by the application, the role trust relationship, the permissions policy, the exact bucket and object ARN, and any bucket policy. Verify encryption settings and whether the role is permitted to use the relevant KMS key. Check organization policies, permission boundaries, endpoint policies, and explicit Deny statements. Confirm that the object key and Region are correct.

A successful console read by an administrator does not prove that an application role has permission. Use temporary credentials and inspect the actual caller identity when possible. Avoid resolving the issue by granting unrestricted S3 access; instead, scope permissions to the required bucket or prefix and actions.

16.3 A queue backlog is growing

A growing queue can indicate that producers are faster than consumers, consumers are failing, or downstream dependencies are slow. Inspect message age, visible and in-flight message counts, receive/delete rates, error logs, concurrency, and downstream throttling. Increasing consumer capacity may help only if downstream systems can handle the additional load.

Check visibility timeout, retry loops, poison messages, and dead-letter queue activity. If processing is not idempotent, increasing retries can duplicate side effects. The correct response may be to fix a failing dependency, quarantine malformed messages, adjust concurrency, or apply backpressure rather than simply adding more workers.

16.4 Cloud spend unexpectedly increases

Start by identifying the service, Region, account, and usage type responsible for the increase. Compare current usage with a prior period, inspect Cost Explorer and detailed billing data, and review recent deployments or traffic changes. Look for idle compute, unexpected data transfer, oversized database capacity, excessive log ingestion, forgotten snapshots, and unbounded serverless invocations.

Cost alerts are most useful when linked to an owner and an investigation procedure. Do not delete resources solely because they look unused; verify ownership, dependencies, retention rules, and recovery requirements. After remediation, monitor the next billing period to confirm that the underlying cause has been addressed.

17. Reference Glossary and Key Distinctions

17.1 Core terms

Availability Zone (AZ): an isolated location within an AWS Region, designed to reduce correlated infrastructure failure. Region: a separate geographic area containing multiple AZs. VPC: a logically isolated virtual network. Subnet: an IP-address range within a VPC associated with one AZ. Security group: a stateful virtual firewall for supported network interfaces. Network ACL: a stateless subnet-level traffic filter.

IAM role: an identity assumed to obtain temporary credentials. IAM policy: a JSON permissions document. SCP: an AWS Organizations policy that sets a maximum permission boundary for member accounts but does not grant permissions. KMS: a managed key service. CloudTrail: an account activity and API event logging service. CloudWatch: a monitoring service for metrics, logs, alarms, and related operational data.

17.2 Storage and compute distinctions

S3 is object storage; EBS is block storage; EFS is shared file storage. EC2 provides virtual machines; ECS and EKS orchestrate containers; Fargate supplies serverless compute for supported containers; Lambda runs event-triggered or directly invoked functions in a managed execution environment. These options differ in control, operational responsibility, scaling model, runtime constraints, and pricing.

RDS manages supported relational database engines. DynamoDB is a managed NoSQL key-value and document database. Redshift is a data warehouse for analytics. Athena queries data in sources such as S3 using SQL. ElastiCache provides managed in-memory caching. Choosing among them requires matching the service to the data model, access pattern, latency, consistency, operations, and cost requirements.

17.3 Messaging distinctions

SQS queues messages for consumers and can buffer work. SNS publishes messages to subscribers and supports fan-out. EventBridge routes events based on rules and event content. Step Functions orchestrates workflows across steps. They can be combined, but each solves a different coordination problem. Queue visibility timeout, delivery semantics, retry rules, and idempotency affect reliability.

17.4 Reliability and cost terms

RTO is the target time to restore service; RPO is the target tolerable data-loss window. High availability focuses on keeping a service available through failures; durability focuses on preserving data. Rightsizing matches provisioned resources to observed needs. FinOps combines finance, engineering, and business perspectives to manage cloud value and spend. IaC defines infrastructure through version-controlled files.

18. Review Checklist and Design Exercises

18.1 Architecture review checklist

Before deployment, verify identity and least privilege; network boundaries and routes; encryption in transit and at rest; secrets handling; logging and alerting; backup retention and restore testing; scaling limits; dependency failure behavior; cost ownership; and documented operational procedures. Confirm which party owns each control under the shared responsibility model.

For every critical workload, record service objectives, RTO, RPO, data classification, expected traffic, capacity assumptions, scaling triggers, alert thresholds, deployment rollback procedure, and disaster recovery method. Validate that quotas, service availability, and required features match the target Region. Run security and recovery tests before treating the architecture as production-ready.

18.2 Design exercises

Exercise A: A news website has unpredictable traffic spikes and serves many static images. Propose a design using a CDN, load balancing, elastic compute, durable object storage, and monitoring. Explain how you would prevent private user-specific responses from being cached publicly.

Exercise B: A financial reporting system needs SQL analytics over historical files, with a goal of controlling query cost. Compare S3 plus Athena with a dedicated warehouse such as Redshift. Discuss partitioning, data format, freshness, concurrency, governance, and the point at which a warehouse may be justified.

Exercise C: A document processor receives duplicate events and sometimes fails because a downstream API is throttled. Explain how you would combine a queue, retries, a dead-letter queue, idempotency, concurrency limits, and monitoring. State which metrics would distinguish a temporary burst from a persistent processing failure.

Exercise D: A business requires an RTO of one hour and an RPO of five minutes for a critical application. Compare backup-and-restore, warm standby, and active/active approaches. Identify what must be replicated, which actions should be automated, and what recovery tests would demonstrate that the targets are achievable.

Appendix A. RAG Evaluation Questions

Use these questions after indexing the PDF. The abstention questions deliberately request information not supplied by this guide; a grounded system should say that the document does not specify the answer and, where appropriate, request current external data.

Q01 · Direct retrieval

What are the five commonly cited characteristics of cloud computing?

Q02 · Direct retrieval

Which AWS service provides object storage, and what is the name of the identifier for an object within a bucket?

Q03 · Direct retrieval

What is the difference between an IAM role's trust policy and its permissions policies?

Q04 · Direct retrieval

Which two AWS services are named as examples of a data warehouse and serverless SQL querying over S3?

Q05 · Comparison

Compare security groups and network ACLs in terms of statefulness and where they apply.

Q06 · Comparison

Explain the difference between RDS Multi-AZ deployments and read replicas.

Q07 · Comparison

Compare SQS, SNS, and EventBridge by their main purpose.

Q08 · Comparison

When would EBS be a better fit than EFS, and when would EFS be a better fit than EBS?

Q09 · Multi-hop

A private EC2 instance must download software updates without accepting unsolicited inbound internet connections. Which networking component is relevant, and what additional routing condition is needed?

Q10 · Multi-hop

A web application uses an ALB and instances across two AZs, but the database is single-AZ. What availability gap remains?

Q11 · Multi-hop

An application role can call s3:GetObject in its identity policy but receives AccessDenied. Name at least four other policy or configuration areas to inspect.

Q12 · Multi-hop

A queue's visible message count is rising and users report stale results. Which additional queue metric and consumer/dependency signals would help diagnose the problem?

Q13 · Architecture

Design a high-level document-processing flow using S3, a queue, compute, durable result storage, and failure handling.

Q14 · Architecture

For a public website with many static images, explain how CloudFront, S3, a load balancer, and elastic compute could work together.

Q15 · Reliability

Define RTO and RPO and explain why frequent backups alone do not prove that both objectives can be met.

Q16 · Security

Explain the AWS shared responsibility model using EC2 and S3 as contrasting examples.

Q17 · Cost

A cloud bill increases unexpectedly. Describe a sequence of investigations before deleting any resources.

Q18 · Abstention test

What is the exact current hourly price of an m7g.large EC2 instance in a specific Region?

Q19 · Abstention test

What is the current service quota for concurrent Lambda executions in this account?

Q20 · Abstention test

Which AWS Region does the fictional company in this document actually use in production?

Q21 · Multi-hop

A team wants an RTO of one hour and an RPO of five minutes. Which disaster recovery strategy is guaranteed to meet both objectives? Explain what information is missing.

Q22 · Policy reasoning

Can an SCP grant a user permission to delete an S3 bucket if no identity policy allows the action? Why or why not?

Q23 · Deployment

Describe one benefit and one risk of a canary deployment, and name a metric that could guide promotion.

Q24 · Data engineering

How can partitioning and columnar file formats affect Athena query cost?

Q25 · Caching

What failure mode can happen when many requests miss the same popular cache key at once, and what does TTL control?

Appendix B. Answer Key and Evidence Notes

These concise expected answers are intended for evaluator use. For a production RAG benchmark, keep this appendix outside the indexed corpus if you want to test answers without exposing the answer key to retrieval.

A1. On-demand self-service, broad network access, resource pooling, rapid elasticity, and measured service.

A2. Amazon S3; the object key identifies the object within the bucket.

A3. The trust policy specifies who may assume a role; permissions policies specify what the assumed role can do.

A4. Amazon Redshift is the data warehouse example; Amazon Athena queries data such as files in S3 using SQL.

A5. Security groups are stateful and attach to supported network interfaces. Network ACLs are stateless and apply at the subnet boundary.

A6. Multi-AZ is primarily for availability/failover; read replicas are primarily for read scaling and certain replication/reporting scenarios. Replication lag may occur.

A7. SQS buffers messages for consumers; SNS publishes/fans out messages; EventBridge routes events based on rules and event content.

A8. EBS fits block-device needs often associated with an instance; EFS fits shared hierarchical file access from multiple supported clients.

A9. A NAT gateway is relevant. The private subnet route table needs an appropriate default route to the NAT gateway, and the NAT gateway itself needs a route to an internet gateway in a public subnet.

A10. The single-AZ database remains a potential availability bottleneck and can cause an outage even if the web and application tiers span AZs.

A11. Possible checks: bucket policy, explicit Deny, permissions boundary, SCP, KMS key policy/permissions, endpoint policy, wrong ARN/key/Region, and actual caller identity.

A12. Check age of oldest message, visible/in-flight counts, receive/delete rates, consumer errors and concurrency, downstream throttling/latency, visibility timeout, retries, and dead-letter queue activity.

A13. One possible flow: upload to S3, enqueue an event/message, process with Lambda or a container, validate input, write results to S3 or a database, retry transient failures, route repeated failures to a DLQ, and notify the user.

A14. CloudFront caches suitable static assets; S3 stores image objects; Route 53 can resolve the domain; an ALB routes dynamic requests to an Auto Scaling application tier. Cache policies must avoid exposing personalized responses.

A15. RTO is target restoration time; RPO is the target tolerable data-loss window. Restore duration, consistency, access to backups/keys, and tested procedures matter in addition to backup frequency.

A16. AWS secures underlying infrastructure. With EC2, the customer also manages guest OS and application configuration; with S3, AWS manages underlying service infrastructure while the customer controls data, permissions, and configuration.

A17. Identify the cost increase by service, Region, account, and usage type; compare periods; review recent changes, compute, transfer, databases, logs, snapshots, and invocations; verify ownership and retention before removing resources.

A18. The guide does not provide current regional EC2 pricing. Check the current AWS pricing source for the exact Region, operating system, tenancy, and purchasing option.

A19. The guide does not state this account's current quota. Check Service Quotas and the account/Region-specific Lambda quota.

A20. The document does not identify a fictional company's actual production Region; it provides a hypothetical European example only.

A21. No strategy is guaranteed from the stated targets alone. Required details include workload architecture, data consistency, dependencies, replication lag, recovery automation, budget, and tested failover performance.

A22. No. An SCP sets a maximum permissions boundary and does not grant permissions. An applicable identity or resource policy must allow the action, subject to other controls.

A23. A canary limits exposure and can enable fast rollback, but it adds routing and analysis complexity and may miss rare failures. Error rate, latency, or a business metric can guide promotion.

A24. Partitioning can let Athena skip irrelevant data when queries filter on partition keys; columnar formats can reduce scanned bytes by reading only needed columns and often compressing data efficiently.

A25. A cache stampede can occur when many requests simultaneously miss a popular key. TTL controls how long a cached entry remains before expiration.

End of guide. Treat this document as a learning and RAG-testing corpus, not as a substitute for current AWS service documentation, security review, or account-specific pricing and quota data.

Appendix C. AWS Service Selection Matrix

Category	Service	Typical fit	Key design concern
Compute	EC2	Virtual machines with broad OS and runtime control	Guest OS patching, capacity planning, instance lifecycle
Compute	Lambda	Event-driven code with managed execution	Runtime and execution limits, retries, concurrency
Containers	ECS / EKS	Containerized services and orchestration	Image security, scheduling, networking, observability
Object storage	S3	Objects, files, backups, static assets, data lakes	Bucket policy, lifecycle, versioning, request patterns
Block storage	EBS	Block devices attached to supported compute	Volume type, snapshots, attachment constraints
File storage	EFS	Shared file system for multiple clients	Throughput mode, mount access, permissions
Relational DB	RDS	Managed relational database engines	Schema, query performance, backup and failover settings
NoSQL	DynamoDB	Key-value/document access patterns	Partition key, indexes, capacity, consistency
Analytics	Athena	SQL over data sources such as S3	Scanned bytes, partitioning, file format, query patterns
Messaging	SQS	Buffer work for asynchronous consumers	Visibility timeout, retries, duplicates, DLQ

Appendix D. Mini Case Study: Online Retailer

A retailer serves product pages, accepts orders, processes payments through an external payment provider, and creates shipment requests. Product images are static, but prices and customer-specific carts are dynamic. The retailer expects large traffic spikes during promotions and must avoid exposing one customer's cart to another customer.

A reasonable starting architecture is Route 53 for DNS, CloudFront for suitable static assets, S3 for product images, an Application Load Balancer for web/API traffic, and a multi-AZ application tier using Auto Scaling or containers. A managed relational database may hold transactional orders, while SQS can buffer fulfillment work. IAM roles should grant services only the access they need, and CloudWatch should monitor latency, errors, queue age, and database health.

Cache rules must distinguish public product imagery from personalized responses. Caching a cart or account response under an overly broad cache key can leak private data. Order creation should use idempotency controls so retries do not create duplicate purchases. Payment-provider timeouts should be handled as uncertain outcomes rather than assumed failures; reconciliation may be needed.

RAG test prompts: Which components serve static images? Why should cart responses not be cached publicly? Which service can buffer fulfillment work? What does idempotency protect against? Which metrics can reveal a fulfillment backlog? Which part of the architecture remains a risk if the database is deployed in only one AZ?

Appendix E. Mini Case Study: Research Data Lake

A research group receives CSV exports every day, sensor files throughout the day, and occasional corrected historical data. Analysts need SQL access, while data stewards need lineage, retention, and controlled access. The group wants to keep raw data available for reprocessing and avoid paying to scan irrelevant data for common date-filtered queries.

S3 can store raw, validated, and curated zones using separate prefixes or buckets. A catalog records table metadata, and a transformation workflow can validate schema, standardize timestamps, remove duplicates, and write columnar files partitioned by date where that partitioning matches query patterns. Athena can query suitable datasets directly. More demanding, highly concurrent warehouse workloads may justify a dedicated warehouse such as Redshift.

Corrections require a policy for whether data is overwritten, versioned, or stored as a new revision. Raw data retention should be explicit. Access should be limited by role and data classification; encryption and audit logs support governance but do not define the business meaning of a field. Data quality checks should report rejected rows and reasons rather than silently dropping them.

RAG test prompts: Why preserve raw files? How can partitioning reduce query work? When might a warehouse be preferable to ad hoc queries? What should happen to rejected rows? Why is encryption not a complete data-governance strategy?

Appendix F. Mini Case Study: Recovery Planning

A booking platform has two workloads. Workload A is a public status page whose temporary outage is inconvenient but not financially severe. Workload B accepts bookings and must preserve confirmed transactions. The organization has limited budget and cannot afford to operate a full active/active environment for every service.

Recovery objectives should be assigned by business impact. The status page may tolerate a longer RTO and be rebuilt from IaC with recent configuration backups. The booking workload may need a shorter RTO and a smaller RPO, stronger database recovery arrangements, and regularly tested restore or failover procedures. Exact choices depend on transaction rate, consistency needs, dependencies, and cost.

A recovery runbook should identify the incident commander, criteria for declaring disaster recovery, required credentials, the recovery Region or environment, data validation steps, traffic-switch procedure, customer communication, and conditions for returning to normal operation. The team should test that encryption keys and required quotas are available in the recovery environment.

RAG test prompts: Why should the two workloads have different recovery objectives? What is the danger of assuming backups are sufficient? Name four operational items that belong in a recovery runbook. Why should a recovery exercise validate data and not just server startup?

Appendix G. Practical Security Review

Identity review: inventory human and workload identities; favor federation and temporary role credentials; require MFA for privileged access; remove unused keys; scope policies to required actions and resources; review role trust policies; and verify that emergency access is documented and monitored.

Network review: map public entry points; keep databases and internal services private where possible; limit ingress to expected ports and sources; inspect route tables, security groups, network ACLs, endpoints, and DNS behavior; and document required outbound paths. Network isolation is useful but should not be treated as the only security control.

Data review: classify sensitive information, define retention, encrypt in transit and at rest where appropriate, restrict key administration, protect backups, and test restore procedures. Review logs for accidental exposure of secrets or personal data. Check whether cache configuration, analytics exports, and test environments expose data beyond its approved audience.

Detection review: ensure CloudTrail collection meets audit needs, CloudWatch alerts have owners, security findings are triaged, and incident-response actions are rehearsed. Detection tools are only effective when people know how to investigate and remediate findings.

Appendix H. Operational Metrics and What They Mean

Metric	Possible interpretation	Caution
Metric	Possible interpretation	Caution
Request latency (p95/p99)	Tail experience for slower requests	Averages can hide slow requests
HTTP 5xx rate	Server-side errors or dependency failures	Correlate with deploys and dependency health
CPU utilization	Compute pressure or inefficient work	Low CPU does not prove an application is healthy
Oldest queue message age	How long work has waited	Interpret alongside arrival and completion rates
Database connections	Connection pressure or leaks	Compare with pool settings and query latency
Cache hit ratio	Share of requests served from cache	High hit ratio can still serve stale data
Athena bytes scanned	Query data volume and cost driver	Inspect partition pruning and file format
Lambda errors/throttles	Function failures or concurrency pressure	Inspect retries and event-source behavior

Appendix I. Benchmark Design for a RAG Pipeline

A useful benchmark set should include several question classes. Direct lookup tests whether a precise fact can be retrieved. Comparison tests whether the system preserves distinctions between similar services. Multi-hop questions require combining facts from different sections. Scenario questions test whether retrieved principles can be applied without inventing details. Unanswerable questions test calibrated abstention.

For each question, define an expected answer, acceptable paraphrases, evidence passage, and scoring criteria before running the model. Score factual correctness separately from citation support. An answer can be correct by coincidence yet unsupported by the retrieved context. Likewise, a correct citation does not rescue a claim that contradicts the source.

For chunking evaluation, compare overlapping and non-overlapping chunks, different chunk sizes, heading-aware splitting, and metadata filters. Keep a fixed question set and report retrieval recall, context precision, answer correctness, groundedness, and abstention quality. Avoid changing the test questions after observing results unless you version the benchmark.

To test chunk boundaries, ask questions whose answer spans a definition and an example in adjacent paragraphs. To test distractor resistance, include several similar services in the same corpus. To test current-information limits, ask for exact live prices, account quotas, or account-specific configuration that the PDF does not contain.

Appendix J. Additional Evaluation Questions

1. A team stores a database password in a Git repository and later deletes the file. Why may the secret still be exposed, and what should the team do?

2. A VPC has three peered VPCs, but a workload in the first VPC cannot reach a workload in the third through the second. Which VPC peering property explains this?

3. An SQS consumer performs a payment operation, crashes before deleting the message, and receives the same message again. What design property reduces the risk of charging twice?

4. A company wants shared file access from multiple EC2 instances. Which storage service is a likely fit, and what access controls still need to be configured?

5. A team lowers S3 storage cost by moving files to an archive class. Which access-pattern and recovery considerations should be checked first?

6. A deployment passes unit tests but causes higher p99 latency after release. Which monitoring and deployment practices can help detect and limit the impact?

7. A team wants to stop all unauthorized actions in member accounts, regardless of identity policies. What can an SCP do, and what can it not do?

8. A dashboard's monthly sales totals disagree with finance. Which data definitions and pipeline-quality checks should be examined?

9. A private subnet uses a NAT gateway but software updates fail. Name routing, security, DNS, or provider-side checks that could explain the failure.

10. A serverless function is throttled during a traffic spike. What should be inspected before simply increasing concurrency?

11. Explain why Multi-AZ database deployment and cross-Region disaster recovery solve different failure scenarios.

12. An application caches user profile responses. What cache-key and authorization risks should be reviewed?

13. A team wants to query a large S3 dataset by date and selected columns. Which file layout and format choices can reduce scanned data?

14. Why does a security finding dashboard not, by itself, prove that an organization is secure?

15. Which facts would you need before recommending an exact AWS instance size or monthly budget?